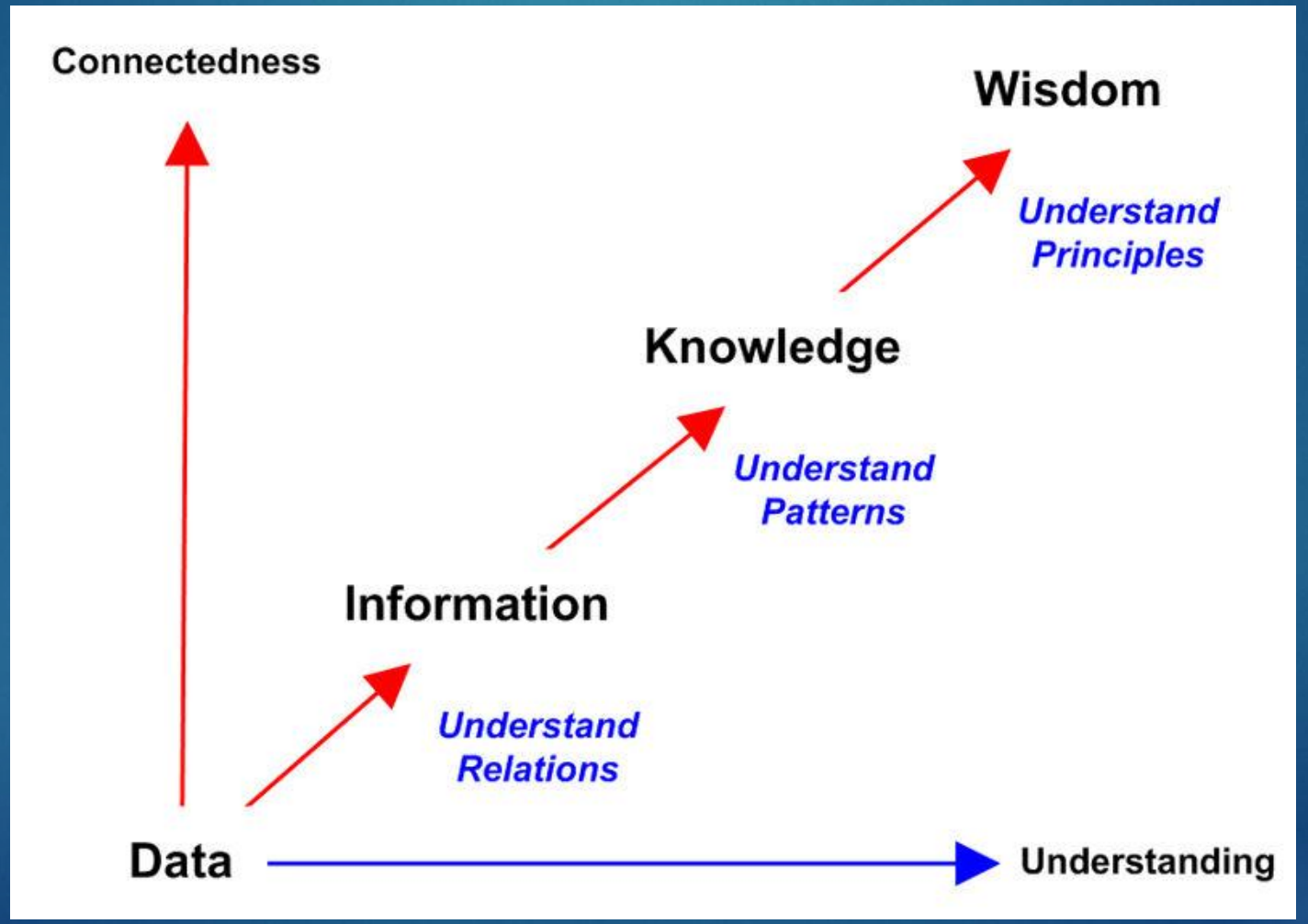
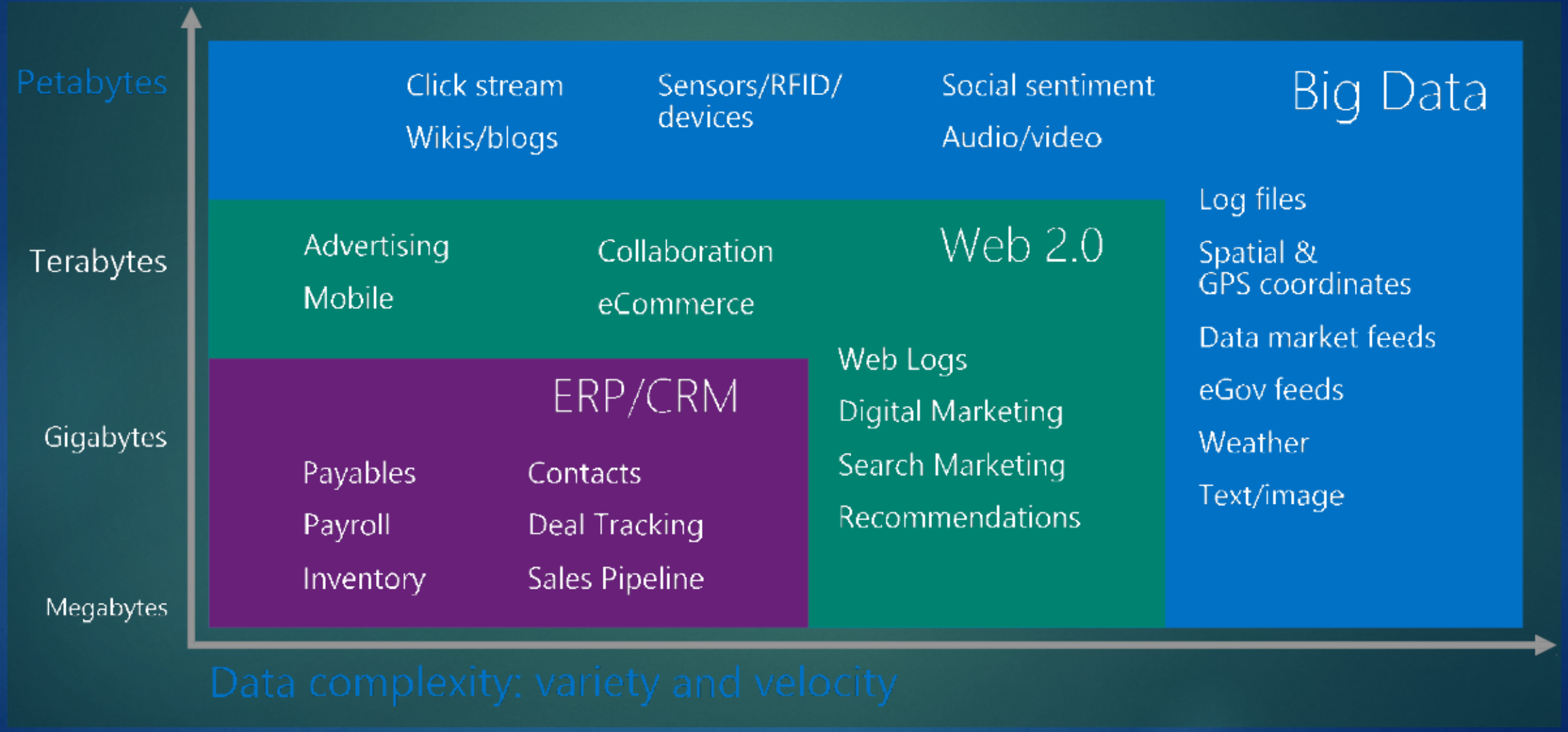
Introduction to Big Data

BigData Hub

Data Concept



Data Evolution



Data Size Vs. Data Processing

- Game to demonstrate this Concept.

Big Data

Demystified

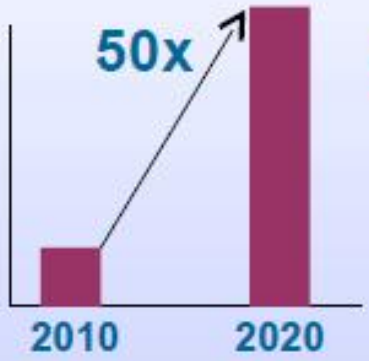
What is big data?

- Big data is a collective term for a set technologies designed for storage, querying and analysis of extremely large data sets, sources and volumes.

- Big data technologies come in where traditional off-the-shelf databases, data warehousing systems and analysis tools fall short.

When to say: I have Big Data?!

Cost efficiently processing the growing Volume



35 ZB

Responding to the increasing Velocity

30 Billion RFID sensors and counting

Collectively analyzing the broadening Variety

80% of the worlds data is unstructured

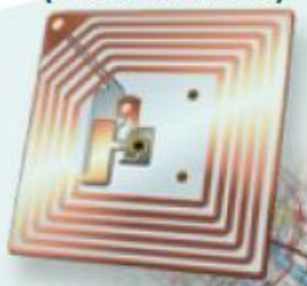
A growing Interconnected and Instrumented World

500+ Million
users posting 55 Million
tweets every day

1.2 Trillion
searches

1+ Billion
active users
spending
700 Million
minutes per
month

30 billion RFID
tags today
(1.3B in 2005)



4.6
billion
camera
phones
world
wide

100s of
millions
of GPS
enabled
devices
sold
annually

76 million smart
meters in 2009…
200M by 2014

2+
billion
people
on the
Web by
end 2011

Intrinsic Property of Data … it grows

90%
of the world's data was created in the last two years

80%
of the world's data today is unstructured

20%
of available data can be processed by traditional systems

1 in 2
business leaders don't have access to data they need

83%
of CIO's cited BI and analytics as part of their visionary plan

5.4X
more likely that top performers use business analytics

How did we end up with so much data?

- **Data Generation:** Human (Internal) ↦ Human (Social) ↦ Machine
- **Data Processing:** Single Core ↦ Multi-Core ↦ Cluster / Cloud
- **An Important Side Note**

  Big Data technologies are based on the concept of clustering - Many computers working in sync to process chunks of our data.

Not just size

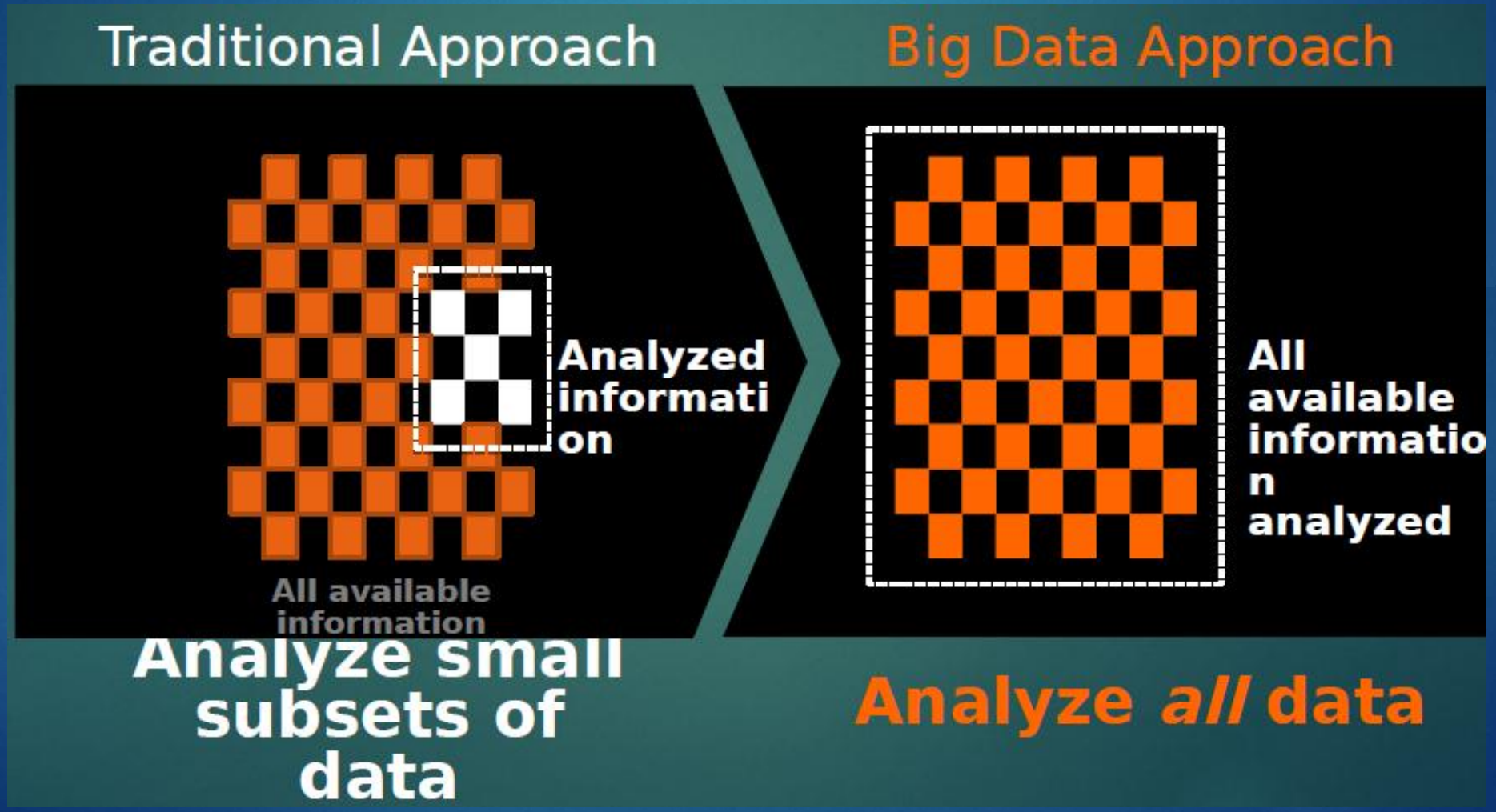
- Big data isn't just about data size, but also about data volume, diversity and inter-connectedness.

Big data is

- Any attribute of our data that challenges either technological capabilities or business needs, like:

- Scaling, moving, storage and retrieval of ever-growing generated data
- Processing many small data points in real-time
- Analysing diverse semi-structured data from multiple sources
- Querying multiple, diverse data sources in real-time

Traditional Vs. Big Data Analysis



The 5 Key Big Data Use Cases

Big Data Exploration

Find, visualize, understand all big data to improve decision making

Enhanced 360° View of the Customer

Extend existing customer views by incorporating additional internal and external data sources

Security/Intelligence Extension

Lower risk, detect fraud and monitor cyber security in real-time

Operations Analysis

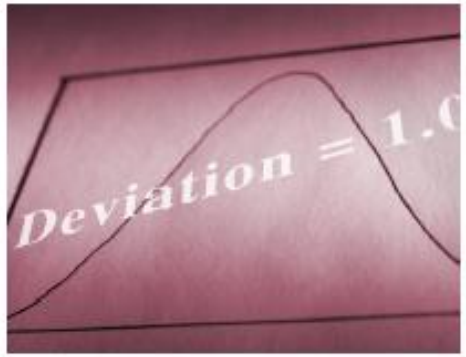
Analyze a variety of machine data for improved business results

Data Warehouse Augmentation

Integrate big data and data warehouse capabilities to increase operational efficiency

More Ways - Wide Ranging Analytics & Techniques

Spatial Analysis



Statistics

Text Analysis

Machine Learning

Temporal Analysis

Audio Analysis

Video Analysis

Image Analysis

Big Data Use cases

- **Healthcare.**
- **Telcos.**
- **Financial services.**
- **Transportation services.**

Hadoop

- The Elephant in the Room

Everyone talks about Hadoop

- Hadoop is a powerful platform for batch analysis of large volumes of both structured and unstructured data.

  From: Conquering Hadoop with Haskell

- **Disk Latency (speed of reads and writes) – not much improvement in last 7-10 years, currently around 70 – 80MB / sec**

How long will it take to read 1TB of data?

1TB (at 80Mb / sec):

1 disk - 3.4 hours
10 disks - 20 min
100 disks - 2 min
1000 disks - 12 sec

Who uses Hadoop?

Hadoop explained

- Hadoop is a horizontally scalable, fault-tolerant, open-source file system and batch-analysis platform capable of processing large amounts of data.

- **HDFS** - Hadoop File System
- **M/R** - Hadoop Map-Reduce platform

Two Key Aspects of Hadoop

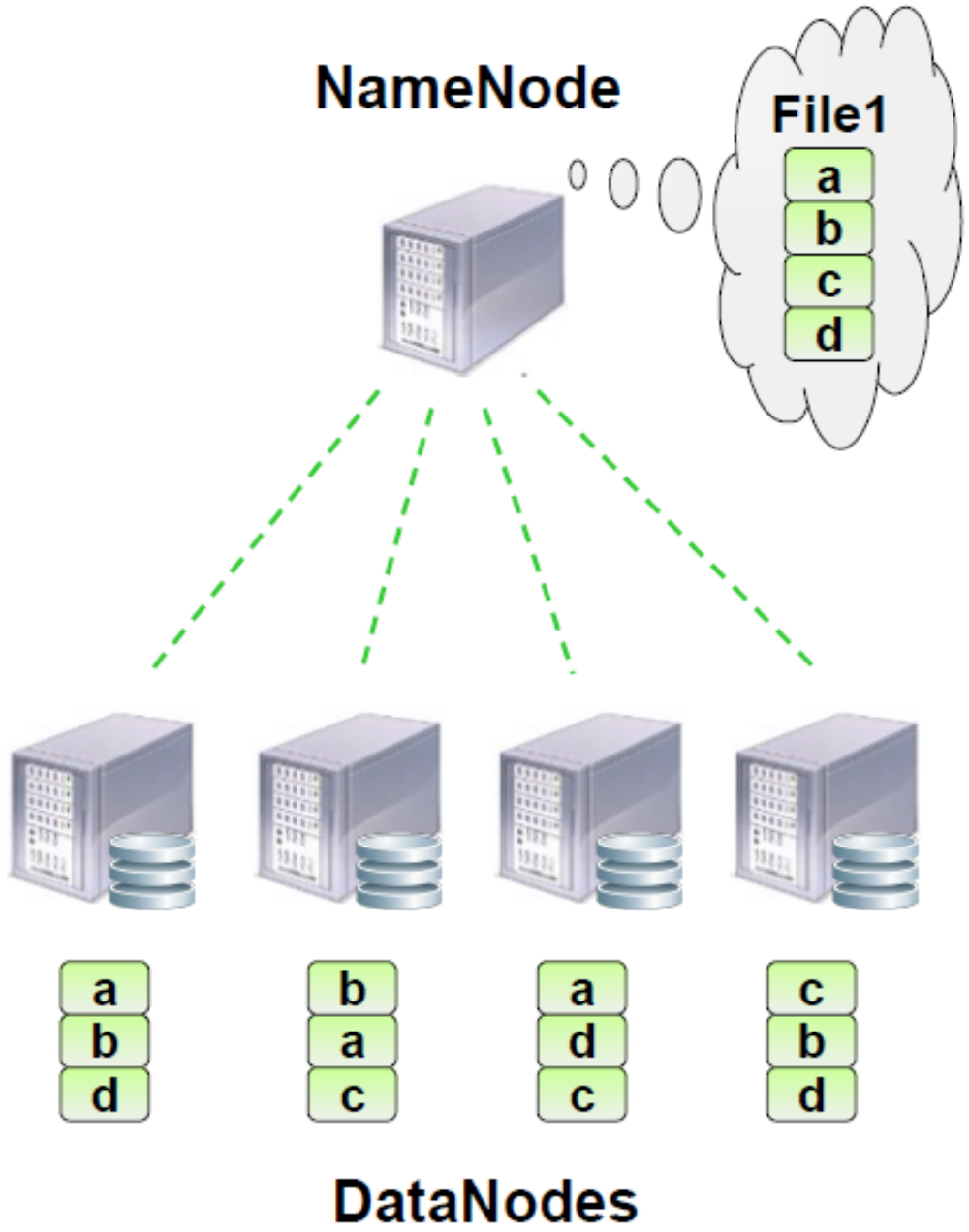
HDFS

Hadoop Distributed File System (HDFS)

- Distributed, scalable, fault tolerant, high throughput
- Data access through MapReduce
- Files split into blocks
- 3 replicas for each piece of data by default
- Can create, delete, copy, but NOT update
- Designed for streaming reads, not random access
- Data locality: processing data on or near the physical storage to decrease transmission of data

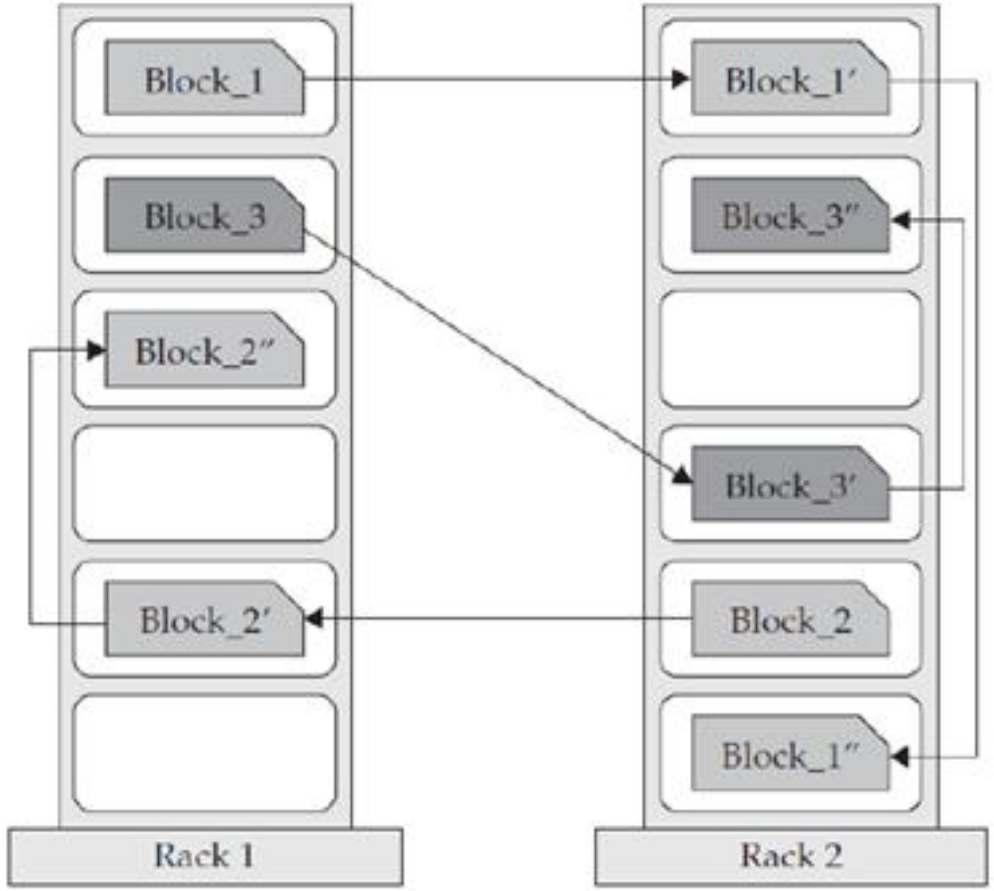
HDFS – Architecture

- **Master / Slave architecture**
- **Master: NameNode**
  - manages the file system namespace and metadata
    - FsImage
    - EditLog
  - regulates client access to files
- **Slave: DataNode**
  - many per cluster
  - manages storage attached to the nodes
  - periodically reports status to NameNode



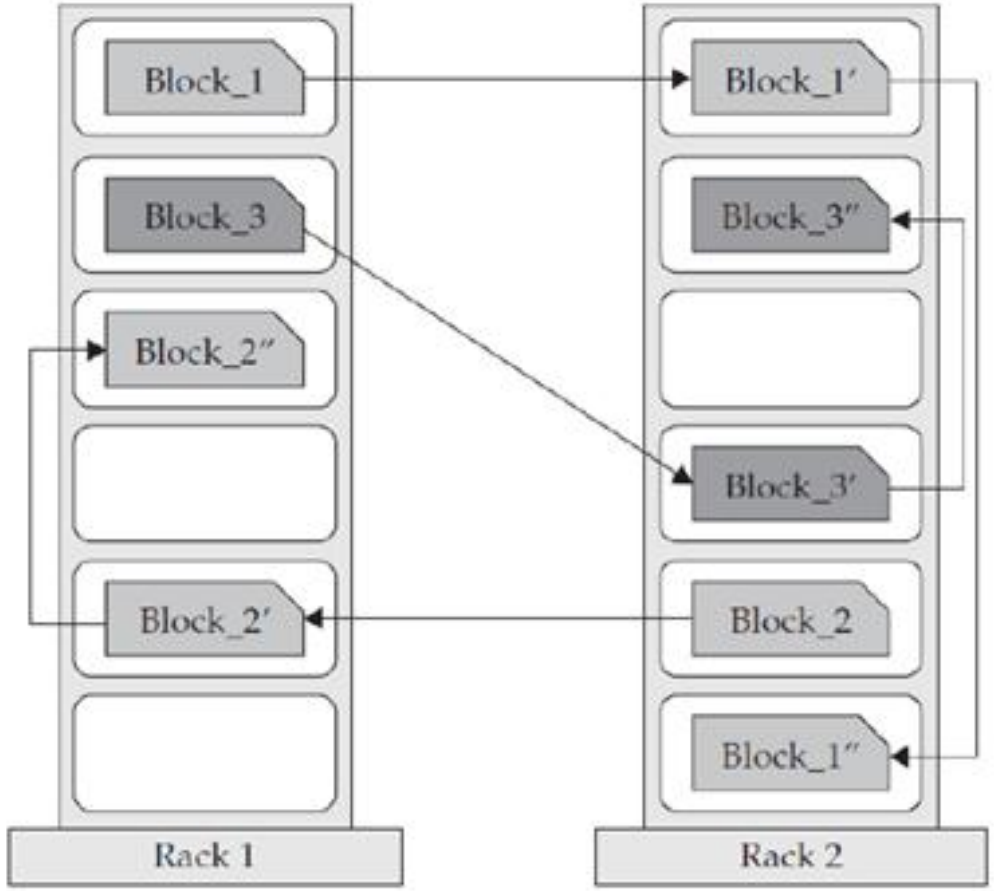
HDFS – Replication

- **Blocks of data are replicated to multiple nodes**
  - Behavior is controlled by replication factor, configurable per file
  - Default is 3 replicas



HDFS – Replication

- **Blocks of data are replicated to multiple nodes**
  - Behavior is controlled by replication factor, configurable per file
  - Default is 3 replicas

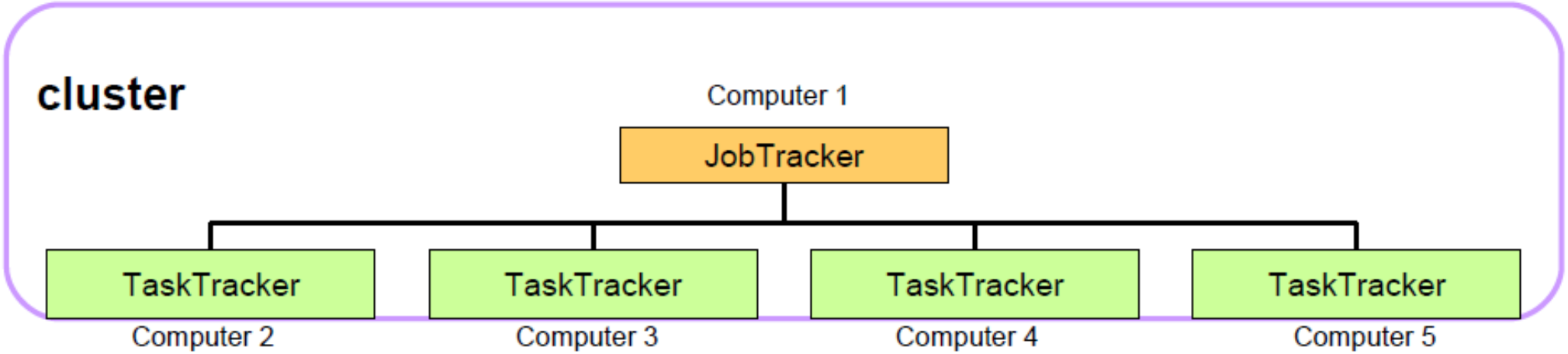


Batch Processing

Hadoop M/R

MapReduce Engine

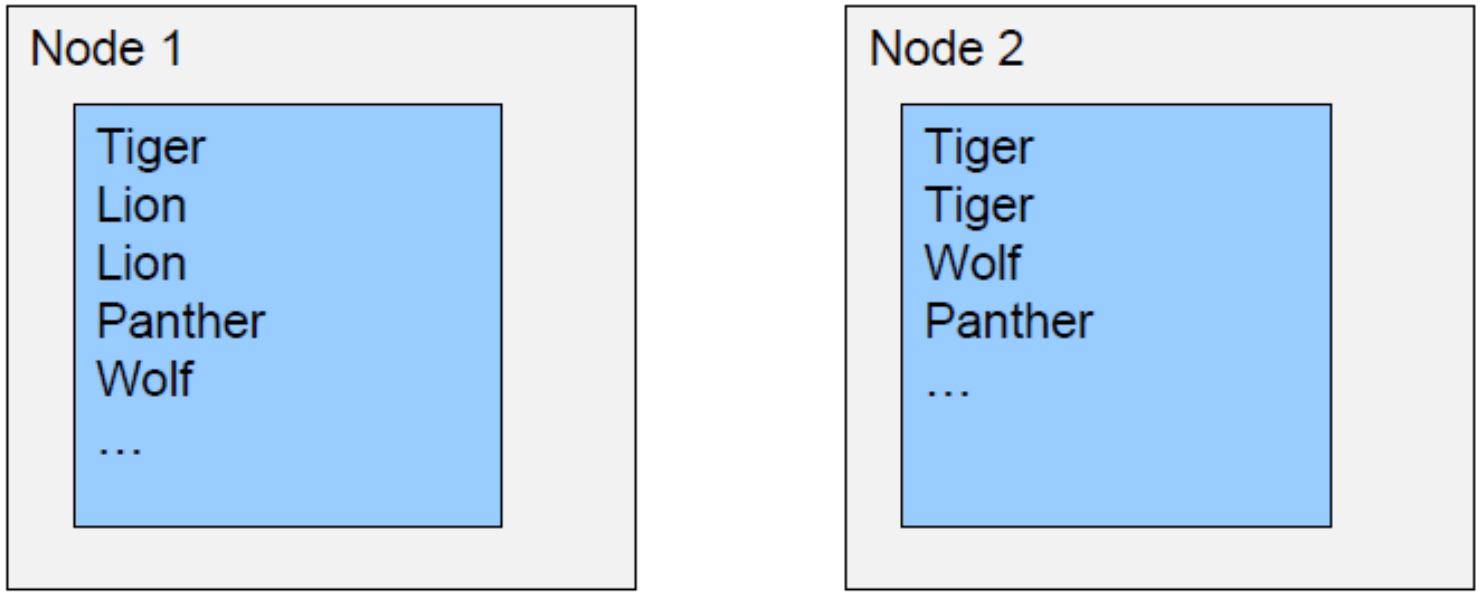
- Master / Slave architecture
  - Single master (JobTracker) controls job execution on multiple slaves (TaskTrackers).
- **JobTracker**
  - Accepts MapReduce jobs submitted by clients
  - Pushes *map* and *reduce* tasks out to TaskTracker nodes
  - Keeps the work as physically close to data as possible
  - Monitors tasks and TaskTracker status
- **TaskTracker**
  - Runs map and reduce tasks
  - Reports status to JobTracker
  - Manages storage and transmission of intermediate output



Word Count Example

- In this example we have a list of animal names
  - MapReduce can automatically split files on line breaks
  - Our file has been split into two blocks on two nodes
- We want to count how often each big cat is mentioned. In SQL that would be:

```
SELECT COUNT(NAME) FROM ANIMALS
       WHERE NAME IN (Tiger, Lion …)
       GROUP BY NAME;
```



Splits

- Files in MapReduce are stored in Blocks (128 MB)
- MapReduce divides data into fragments or **splits**.
  - One map task is executed on each split
- Most files have records with defined **split points**
  - Most common is the end of line character
- The `InputSplitter` class is responsible for taking a HDFS file and transforming it into splits.
  - Aim is to process as much data as possible locally

How do we do this on Hadoop data?

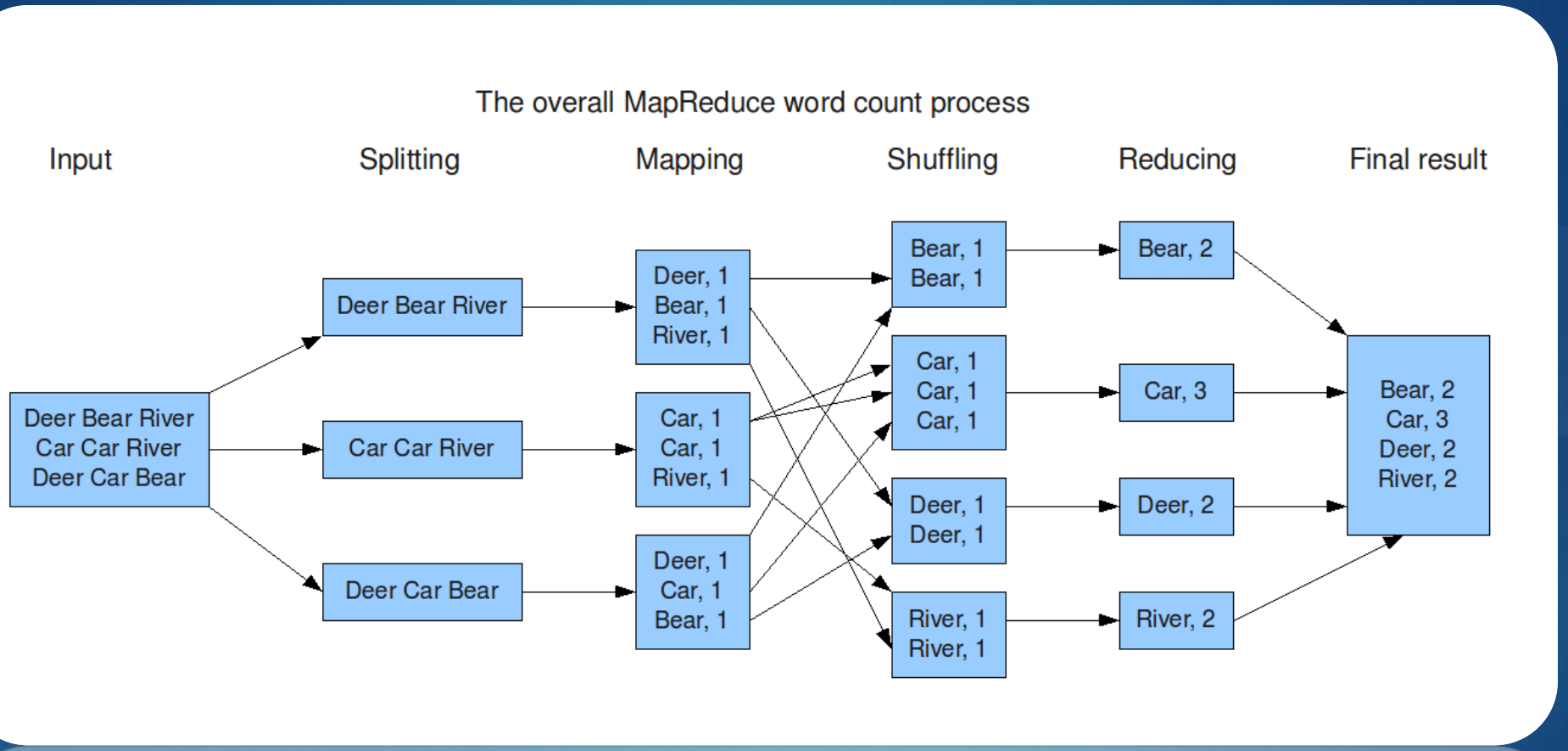
```
INSERT OVERWRITE TABLE xyz_com_page_views
SELECT page_views.*
FROM page_views
WHERE page_views.date >= '2008-03-01' AND page_views.date <= '2008-03-31' AND
      page_views.referrer_url like '%xyz.com';
```

```
INSERT OVERWRITE TABLE pv_users
SELECT pv.*, u.gender, u.age
FROM user u JOIN page_view pv ON (pv.userid = u.id)
WHERE pv.date = '2008-03-03';
```

Source: https://cwiki.apache.org/confluence/display/Hive/Tutorial

Batch processing of large data sets

- Hadoop gives us the basic tools for large data processing in the form of M/R.
  However, Hadoop M/R is pretty annoying to work with directly as it lacks a lot of relevant tools for the job (statistical analysis, machine learning etc.)



Source: http://xiaochongzhang.me/blog/?p=338

Classes

- There are three main classes reading data in MapReduce:
  - `InputSplitter`, dividing a File into Splits
    - normally the block sizes but depends on number of requested Map tasks etc.
  - `RecordReader`, takes a split and reads the files into records
    - for example one record per line (`LineRecordReader`)
  - `InputFormat`, takes each record and transforms it into a <key, value> pair that is then forwarded to the Map task
- Lots of additional helper classes handling compression etc.
  - IBM provides additional compression handlers for lzo etc.

```java
package myPackage;

import org.apache.hadoop.conf.Configuration;

public class SalesDriver {

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        // Use programArgs array to retrieve program arguments.
        String[] programArgs = new GenericOptionsParser(conf, args)
                .getRemainingArgs();
        Job job = new Job(conf);
        job.setJarByClass(SalesDriver.class);
        job.setMapperClass(SalesMapper.class);
        job.setReducerClass(SalesReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        // TODO: Update the input path for the location of the inputs of the map-reduce job.
        FileInputFormat.addInputPath(job, new Path("[input path]"));
        // TODO: Update the output path for the output directory of the map-reduce job.
        FileOutputFormat.setOutputPath(job, new Path("[output path]"));

        // Submit the job and wait for it to finish.
        job.waitForCompletion(true);
        // Submit and return immediately:
        // job.submit();
    }
}
```

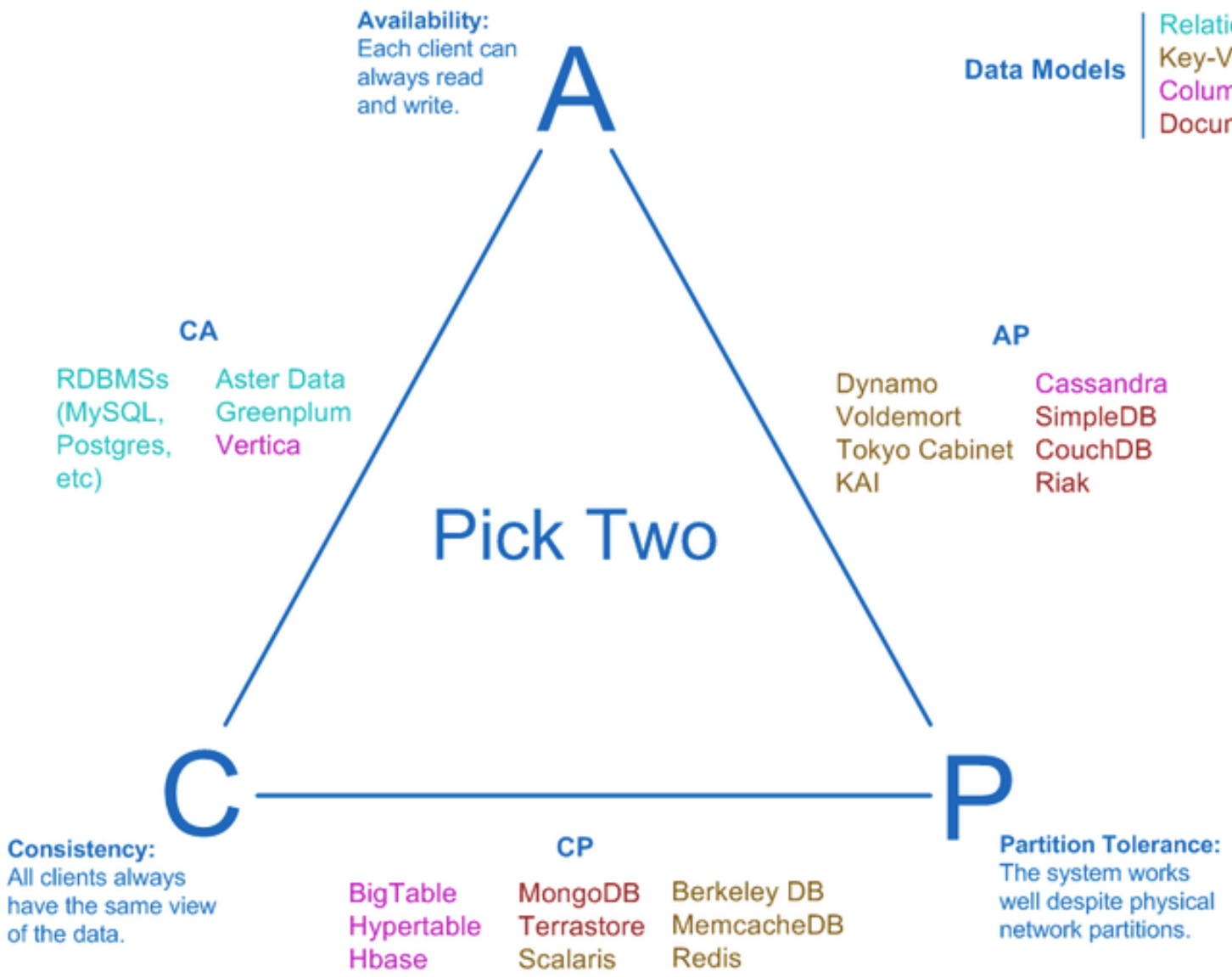
```java
package myPackage;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class SalesMapper extends Mapper<LongWritable, Text, Text, IntWritable> {

    @Override
    public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {
    }
}
```

```java
package myPackage;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class SalesReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
    }
}
```

Databases

- In the big data world

Visual Guide to NoSQL Systems



Real-Time

- Big Data Now!

Real-Time big data processing

- Processing big data in real-time is about data volumes rather than just size. For example, given a rate of 100K ops/sec, how do I do the following in real-time?:
- Find anomalies in a data stream (spam)
- Group check-ins by geo
- Identify trending pages / topics

Hadoop isn't for real-time processing

- When it comes to data processing and analysis, Hadoop's M/R framework is wonderful for batch (offline) processing.
- However, processing, analysing and querying Hadoop data in real-time is quite difficult.

Apache Storm and Apache Spark

- Apache Storm and Apache Spark are two frameworks for large-scale, distributed data processing in real-time.
- One could say that both Storm and Spark are for real-time data processing what is Hadoop M/R for batch data processing.

Apache Storm - Highlights

- Runs on the JVM (Clojure / Java mix)
- Fully distributed and fault-tolerant
- Highly-scalable and extremely fast
- Interoperability with popular languages (Scala, Python etc.)
- Mature and production ready
- Hadoop interoperability via Storm-YARN
- Stateless / Non-Persistent (Data brought to processors)

Apache Spark - Highlights

- Fully distributed and extremely fast
- Write applications in Java Scala and Python
- Perfect for both batch and real-time
- Combine Hadoop SQL (Shark), Machine Learning and Data streaming
- Native Hadoop interoperability
- HDFS, HBase, Cassandra, Flume as data sources
- Stateful / Persistent (Processors brought to data)

Storm & Spark - Use Cases

- Continuous/Cyclic Computation
- Real-time analytics
- Machine Learning (eg. recommendations, personalisation)
- Graph Processing (eg. social networks) - Only Spark
- Data Warehouse ETL (Extract, Transform, Load)

Practical session

Wrap up your Knowledge...

- 1- Virtualization

Prerequisites Software

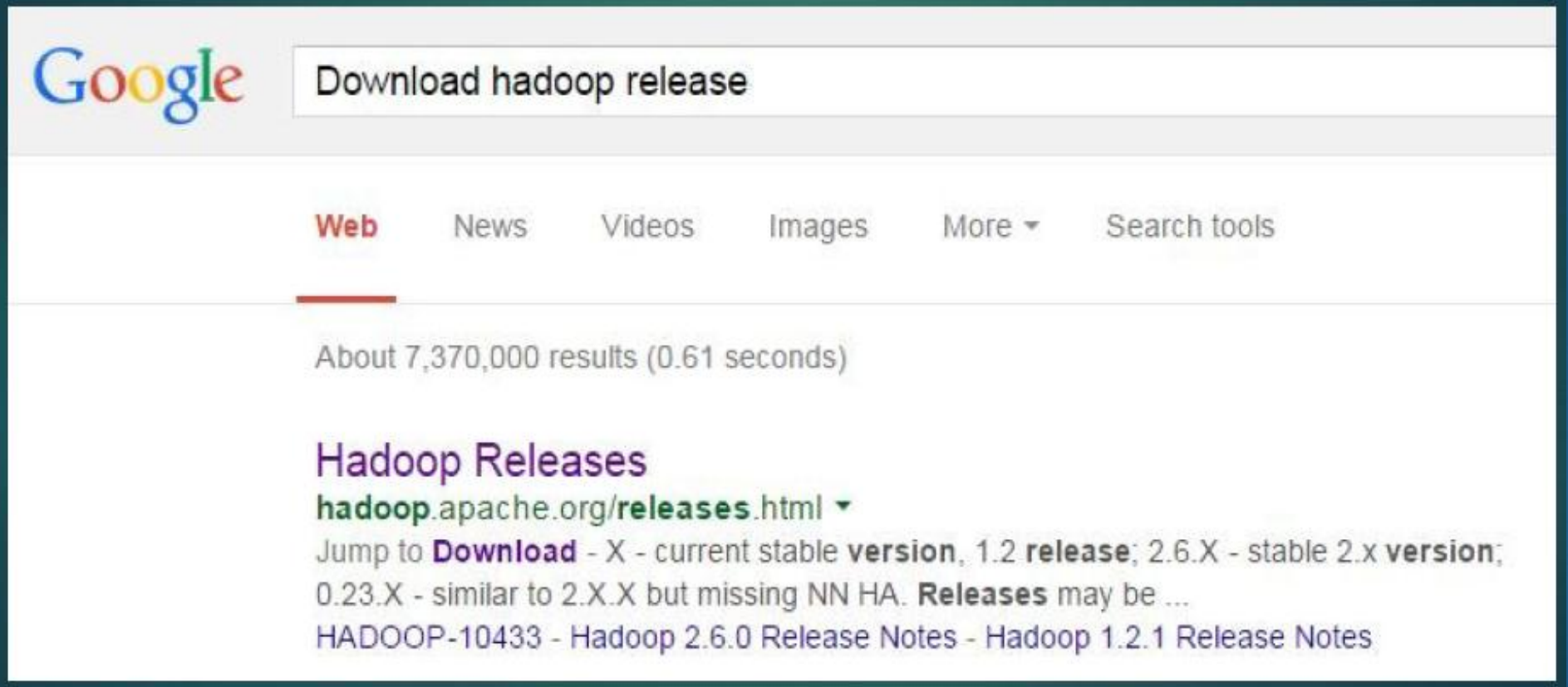
Installing JAVA

- sudo apt-get update
- sudo apt-get install sun-java6-jdk
- Java –version

Install & Configure ssh

- sudo apt-get install openssh-server
- ssh-keygen -t rsa -P "“
- #cat $HOME/.ssh/id_rsa.pub >> $HOME/.ssh/authorized_keys

Hadoop Download & Install



Hadoop Download & Install

Hadoop Download & Install

Download

- **1.2.X** - current stable version, 1.2 release
- **2.6.X** - stable 2.x version
- **0.23.X** - similar to 2.X.X but missing NN HA.

Releases may be downloaded from Apache mirrors.

Download a release now!

On the mirror, all recent releases are available.

Hadoop Download & Install

Hadoop Download & Install

Index of /hadoop/common

Name	Last modified	Size	Description
Parent Directory		-	
current/	30-Dec-2014 00:45	-	
hadoop-1.2.1/	30-Dec-2014 00:44	-	
hadoop-2.5.2/	30-Dec-2014 00:45	-	
hadoop-2.6.0/	30-Dec-2014 00:45	-	
stable/	30-Dec-2014 00:45	-	
stable1/	30-Dec-2014 00:44	-	
stable2/	30-Dec-2014 00:45	-	
readme.txt	06-Nov-2014 07:22	95	

Hadoop Download & Install

Hadoop Download & Install

- In the Linux Terminal, Writ:

wget http://supergsego.com/apache/hadoop/common/hadoop-1.2.1/hadoop-1.2.1-bin.tar.gz

and then hit ENTER.

Editing .bashrc file

- #gedite ~/.bashrc

Add the following lines at the end of the file:

export HADOOP_PREFIX=/opt/hadoop/

export PATH=$PATH:$HADOOP_PREFIX/bin

Hadoop main Installation

- #tar –zxvf hadoop-1.2.1-bin.tar.gz
- #mv /home/hadoop-1.2.1 /opt/hadoop
- #gedite /opt/hadoop/conf/hadoop-env.sh

Editing conf/*-site.xml files

- 1- “Core-site.xml” File:
- #gedit /opt/hadoop/conf/core-site.xml

root@hadoop: /opt/hadoop/conf

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<!-- Put site-specific property overrides in this file. -->

<configuration>
<property>
<name>fs.default.name</name>
<value>hdfs://localhost:10001</value>
</property>
<property>
<name>hadoop.tmp.dir</name>
<value>/opt/hadoop/tmp</value>
</property>
</configuration>
```

Editing conf/*-site.xml files

- 2-”Mapred-site.xml” File
- #gedit /opt/hadoop/conf/mapred-site.xml

root@hadoop: /opt/hadoop/conf

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<!-- Put site-specific property overrides in this file. -->

<configuration>
<property>
<name>mapred.job.tracker</name>
<value>localhost:10002</value>
</property>
</configuration>
```

Editing conf/*-site.xml files

- 3-”hdfs-site.xml” File.
- #gedit /opt/hadoop/conf/hdfs-site.xml

root@hadoop: /opt/hadoop/conf

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>

<!-- Put site-specific property overrides in this file. -->

<configuration>
<property>
    <name>dfs.replication</name>
    <value>1</value>
</property>
</configuration>
```

Formatting Namenode File System

- #hadoop namenode –format

```
15/03/10 23:44:36 INFO namenode.FSEditLog: dfs.namenode.edits.toleration.length
= 0
15/03/10 23:44:36 INFO namenode.NameNode: Caching file names occuring more than
10 times
15/03/10 23:44:37 INFO common.Storage: Image file /opt/hadoop/tmp/dfs/name/curre
nt/fsimage of size 110 bytes saved in 0 seconds.
15/03/10 23:44:39 INFO namenode.FSEditLog: closing edit log: position=4, editlog
=/opt/hadoop/tmp/dfs/name/current/edits
15/03/10 23:44:39 INFO namenode.FSEditLog: close success: truncate to 4, editlog
=/opt/hadoop/tmp/dfs/name/current/edits
15/03/10 23:44:39 INFO common.Storage: Storage directory /opt/hadoop/tmp/dfs/nam
e has been successfully formatted.
15/03/10 23:44:39 INFO namenode.NameNode: SHUTDOWN_MSG:
/************************************************************
SHUTDOWN_MSG: Shutting down NameNode at hadoop/127.0.1.1
************************************************************/
root@hadoop:/opt/hadoop/conf#
```

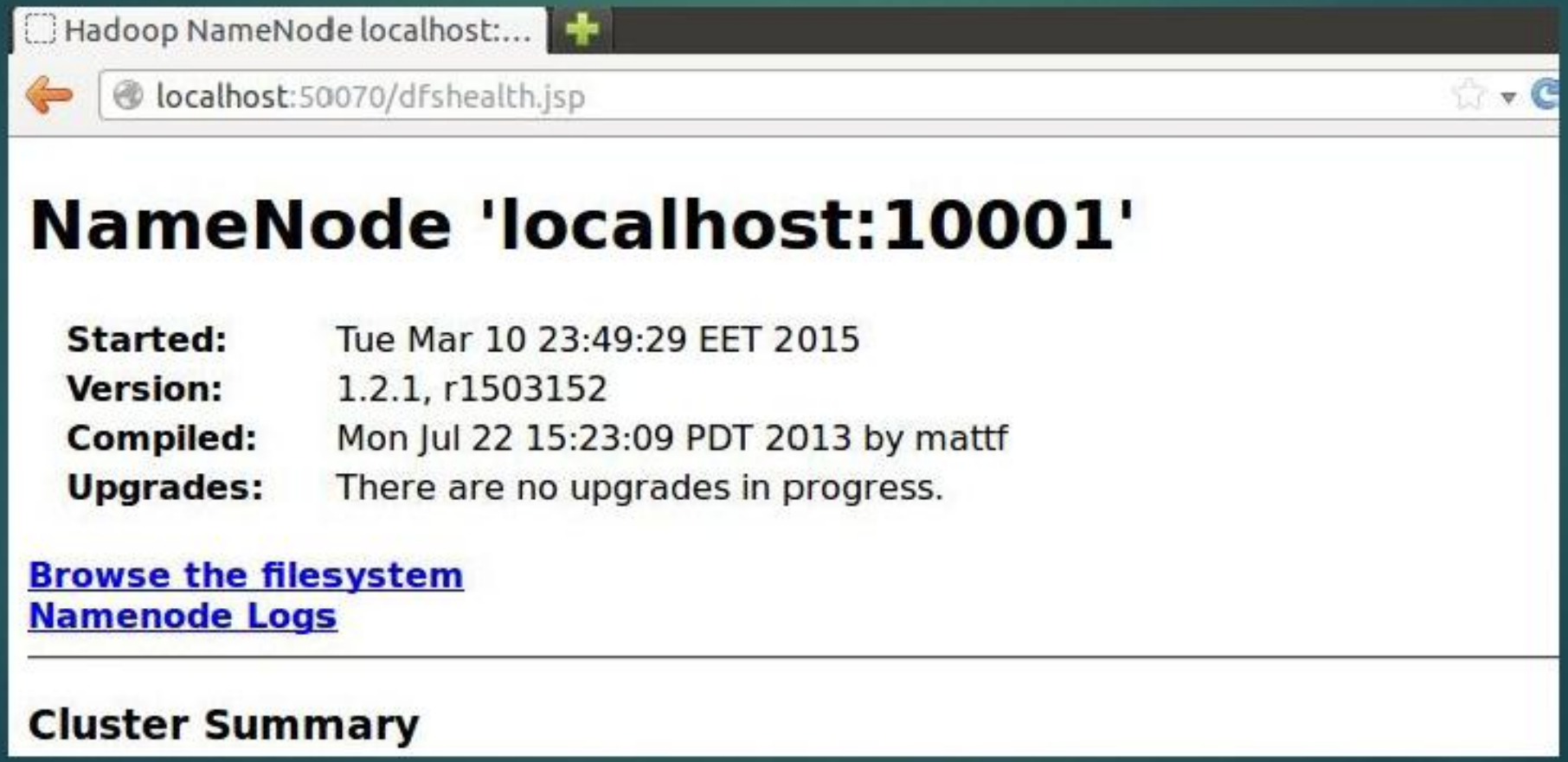
Firing Hadoop Deamons

- #start-all.sh

```
root@hadoop:/opt/hadoop/conf# start-all.sh
starting namenode, logging to /opt/hadoop/libexec/../logs/hadoop-root-namenode-h
adoop.out
localhost: starting datanode, logging to /opt/hadoop/libexec/../logs/hadoop-root
-datanode-hadoop.out
localhost: starting secondarynamenode, logging to /opt/hadoop/libexec/../logs/ha
doop-root-secondarynamenode-hadoop.out
starting jobtracker, logging to /opt/hadoop/libexec/../logs/hadoop-root-jobtrack
er-hadoop.out
localhost: starting tasktracker, logging to /opt/hadoop/libexec/../logs/hadoop-r
oot-tasktracker-hadoop.out
root@hadoop:/opt/hadoop/conf#
```

Testing Installation

- Localhost:50070



yosufessa@gmail.com

Mot.

Sel.